



Actividad 6

Serialización y Networking I

Entrega

- **Lugar:** Repositorio personal de GitHub — Carpeta: Actividades/AC6
- **Fecha máxima de entrega:** 7 de Mayo 17:20
- **Ejecución de actividad:** La Actividad será ejecutada **únicamente** desde la terminal del computador. Los *paths* relativos utilizados en la Actividad deben ser coherentes con esta instrucción, y no pueden modificarse.

Importante: Antes de comenzar, comprueba que Git este funcionando correctamente en tu repositorio privado. Para esto, **sube los archivos base de la actividad de inmediato** (*add*, *commit*, *push*). Se espera que en esta actividad (así como en las demás actividades y tareas) utilices Git a lo largo de **todo tu desarrollo** como una herramienta, no sólo como un método de entrega. Es por esto que recomendamos enfáticamente que vayas subiendo tus cambios constantemente (*push*), ya que **problemas de último minuto** relacionados con la entrega y Git **no serán considerados**.

Introducción: Lienzo

Lienzo es la nueva página web que los alumnos del DCC hagan controles de alternativas. Se te pidió a ti que implementes las operaciones básicas de Lienzo.

Archivos

El código base tiene 3 archivos. Debes subir los archivos con el tag **Entregar** al repositorio para que sean evaluados. Los cambios en los otros archivos no serán considerados y no es necesario que los subas al repositorio.

- **Modificar** **Entregar** `servidor.py`: Contiene la implementación del **servidor** utilizando `flask`. Aquí deberás completar los métodos que procesan los *requests* del cliente.
- **Modificar** **Entregar** `cliente.py`: Contiene la implementación del **cliente**, es el encargado de comunicarse con el servidor mediante *requests* HTTP.
- **Modificar** `menu.py`: Contiene una interfaz de consola ya implementada que permite: Listar controles disponibles; seleccionar un control; responder preguntas; y enviar respuestas al servidor.

Puedes modificar este archivo para probar tu código, pero no será evaluado.

Operaciones de Lienzo

Deberás implementar las siguientes operaciones tanto a nivel de servidor como de cliente:

- Obtener lista de controles.
- Obtener detalles del un control.
- Subir respuestas de un control.

Se recomienda completar 1 *endpoint* a la vez completando tanto el servidor como el cliente de cada uno antes de pasar al siguiente *endpoint*.

Parte 1. Obtener lista de controles

Para esta sección se usan los siguientes métodos:

- `servidor.py`:

- **No modificar** `def get_lista_de_controles() -> Response:`

Maneja *requests* GET al *endpoint* “/controles”.

Retorna un **Response** con los nombres de los archivos de controles disponibles en la carpeta `controles` ordenados alfabéticamente en formato JSON.

Este método ya está completo por lo que no debes modificarlo. Sin embargo, debes completar el método `get_controles` en el archivo `cliente.py` para llamar al *endpoint* correctamente.

- `cliente.py`:

- **Modificar** `def get_controles() -> tuple[int, list]:`

Realiza un *request* GET al *endpoint* “/controles”.

Debe retornar una tupla con:

- El código de estado HTTP (`int`).
- La lista de controles obtenida desde el servidor, o una lista vacía si el resultado del request no es 200.

Parte 2. Obtener detalles de un control

- `servidor.py`:

- **Modificar** `def get_control(nombre_control) -> Response:`

Maneja *requests* GET al *endpoint* “/controles/<nombre_control>”.

Recibe el nombre de un control y debe retornar un **Response** con la información de este control en un diccionario en formato JSON leyendo el archivo correspondiente desde la carpeta `controles` con la librería `json`.

Si el archivo correspondiente al control no existe, debe retornar un **Response** con `status = 404` y un mensaje con el detalle del error en JSON:

```
{"Mensaje": f"Control no encontrado: {nombre_control}"}
```

■ cliente.py:

- **Modificar** `def get_preguntas_control(nombre_control) -> tuple[int, dict]:`

Realiza un *request* GET al *endpoint* “/controles/<nombre_control>”.

Debe retornar una tupla con:

- El código de estado HTTP.
- Un diccionario con el contenido retornado por el servidor.

Parte 3. Enviar respuestas de un control

■ servidor.py:

- **Modificar** `def post_control_answer(nombre_control) -> Response:`

Maneja *requests* POST al *endpoint* “/controles/<nombre_control>”.

Este *endpoint*:

- Recibe el nombre del alumno como parámetro en la URL de la forma “?alumno=<alumno>”.
- Debe revisar si el control `nombre_control` existe. En caso de no existir, debe inmediatamente retornar un `Response` con *status* 404 y mensaje en formato JSON:

```
{"Mensaje": f"Control no encontrado: {nombre_control}"}
```

- Recibe en el *body* del *request* una lista con las respuestas del alumno en formato JSON.
- Debe guardar estas respuestas en un archivo dentro de la carpeta `respuestas` usando la librería `json`. El nombre del archivo de respuestas debe ser:

```
<usuario>-<nombre_control>.json
```

Por ejemplo, si el usuario `alopez7` envía las respuestas del control `00-Entorno de trabajo`, el archivo de respuestas debe ser “`respuestas/alopez7-00-Entorno de trabajo.json`”.

En el caso de que ya exista una respuesta registrada para ese usuario y control, el servidor debe en vez retornar un `Response` con *status* de error 409 (*Conflict*) y con el mensaje:

```
{"Mensaje": f"No se pueden sobrecribir las respuestas"}
```

- Debe retornar un mensaje de confirmación con formato:

```
{"Mensaje": "Respuesta guardada correctamente"}
```

■ cliente.py:

- **Modificar** `def post_respuestas_control(nombre_control: str, respuestas: list[int], alumno: str) -> tuple[int, dict]:`

Realiza un *request* POST al *endpoint* “/controles/<nombre_control>”.

Debe:

- Enviar las respuestas en el *body* del *request*.
- Enviar el nombre del alumno como parámetro en la URL de la forma “?alumno=<alumno>”.
- Retornar una tupla con el código de estado HTTP y la respuesta del servidor.

Ejecución de código

Para hacer funcionar el código debes ejecutar el código del servidor desde la carpeta que contiene los archivos mencionados usando:

```
python3 servidor.py
```

Una vez que el servidor esté funcionando debería imprimir en consola un mensaje que dice **WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.** No te preocupes, ya que este mensaje es normal.

Luego puedes hacer requests al servidor desde la clase cliente usando el menú ejecutando:

```
python3 menu.py
```

Si `python3` no funciona, probar con el comando específico de tu computador. Este puede ser `py`, `python`, `py3` o `python3.12`.

Si modificas el código en `servidor.py`, debes dejar se ejecutar el servidor usando CTRL+C en la consola y luego volver a ejecutarlo para que los cambios sean reflejados.

Notas

- No puedes hacer *import* de otras librerías externas a las entregadas en el archivo.
- Tu actividad será evaluada usando tu archivo modificados `server.py` y `client.py`.
- Recuerda que la ubicación de tu entrega es en **tu repositorio de Git**. En la rama (*branch*) por defecto del repositorio: `main`.
- Se recomienda completar la actividad 1 *request* a la vez trabajando simultáneamente en el servidor y el cliente.
- Recuerda que esta evaluación presenta corrección **automatizada**. Si entregas un código que se cae al momento de correr los *tests*, será evaluado con 0 puntos.
- Siéntete libre de agregar nuevos `print` en cualquier lugar para revisar objetos, modificaciones, etc... Es una herramienta muy útil para encontrar errores. Y si aparece un error inesperado, ¡léelo! Intenta interpretarlo.

Objetivo de la actividad

- Aplicar conocimientos de Serialización mediante el uso de la librería `json`.
- Aplicar conceptos de la arquitectura cliente-servidor usando `flask`.
- Probar código mediante la ejecución de *test* y usando `menu.py`.

Ejecución de *tests*

En esta actividad se provee de varios archivos `.py` los cuáles contiene diferentes *tests* que ayudan a validar el desarrollo de la actividad. Para ejecutar estos *tests*, **primero debes posicionar tu terminal/consola en la carpeta de la actividad (Actividades/AC6)**. Luego, desde esta misma, debes escribir el siguiente comando para ejecutar todos los *tests* de la actividad:

```
python3 -m unittest discover tests_publicos -v -b
```

Existe una clase de test por cada método a implementar. Si testear ejecutar un método en particular, puedes hacer usando el comando:

```
python3 -m unittest -v -b <carpeta>.<archivo>.<NombreClase>
```

Por ejemplo, si quieres testear el método `get_controles` del cliente:

```
python3 -m unittest -v -b tests_publicos.test_cliente.VerificarGetControles
```

Importante: recuerda que si `python3` no funciona, probar con el comando específico de tu computador. Este puede ser `py`, `python`, `py3` o `python3.12`.